

PQC 마이그레이션 환경에서의 Sandwich 플랫폼 분석 및 개선 방향

송민호*, 엄시우*, 이승원*, 김하경*, 서화정**
* 한성대학교 (대학원생), **한성대학교 (교수)

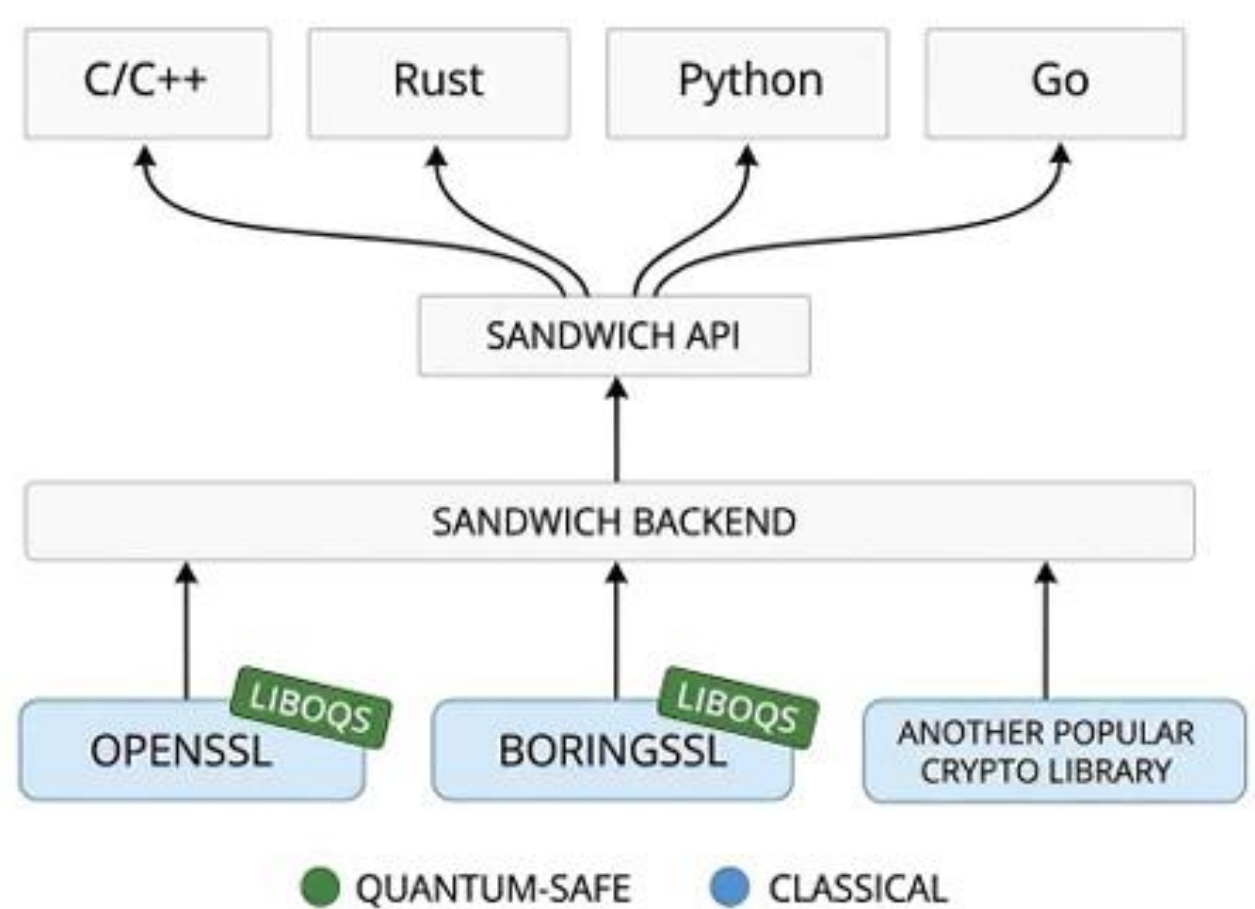
서론

기존 공개키 암호 체계를 대체하기 위해 양자내성암호(Post-Quantum Cryptography, PQC)에 대한 연구와 NIST 중심의 표준화가 활발히 진행되고 있다. 그러나 PQC 전환은 단순한 알고리즘 교체만으로 해결되지 않으며, 기존 시스템과의 호환성, 운영 환경의 다양성, 성능 및 자원 제약 등을 함께 고려해야 한다. 이에 따라 시스템의 안정성과 보안을 유지하면서 암호 알고리즘을 유연하게 적용·교체할 수 있는 crypto-agility가 중요하게 요구된다. SandboxAQ의 Sandwich 플랫폼은 이러한 crypto-agility를 지원하는 암호 프레임워크로, 다양한 암호 라이브러리와 알고리즘을 통합 제공하고 기존 암호와 PQC를 결합한 하이브리드 구성을 지원한다. 이를 통해 점진적인 암호 전환이 가능하지만, 동시에 다양한 프로토콜·알고리즘·라이브러리 조합을 고려해야 하므로 실제 PQC 마이그레이션 과정에서 암호 구성 선택의 복잡성이 증가하는 한계가 있다.

Sandwich 플랫폼

SandboxAQ의 Sandwich 플랫폼은 다양한 암호 알고리즘과 구현을 통합하여 제공하는 오픈소스 암호 프레임워크로 개발자가 단일 인터페이스를 통해 암호 기능을 활용할 수 있도록 지원한다. Sandwich는 통합된 API를 제공하여 개발자가 개별 암호 라이브러리를 직접 다루지 않고도 암호 프로토콜과 알고리즘을 사용할 수 있도록 한다. 또한 OpenSSL, BoringSSL, libOQS 등 다양한 암호라이브러리를 백엔드로 활용하며 이를 통해 기존 암호와 PQC를 모두 지원한다. 특히 Sandwich는 암호 프로토콜과 구현을 조합하여 하나의 Sandwich 객체로 구성하는 구조를 가지며 이를 통해 다양한 암호구성을 유연하게 생성할 수 있다. 개발자는 프로토콜과 구현을 선택하여 원하는 암호 스택을 구성할 수 있으며 이러한 구성은 하나의 통합된 암호객체로 제공된다. 또한 Sandwich는 C/C++, Rust, Python, Go 등 다양한 프로그래밍 언어를 지원하여 서로 다른 개발 환경에서도 일관된 방식으로 암호 기능을 활용할 수 있도록 한다.

이와 같은 구조는 암호 알고리즘 및 라이브러리를 애플리케이션 코드와 분리하여 관리할 수 있도록 하며 코드 수정이나 재컴파일 없이도 암호 구성을 변경할 수 있는 유연성을 제공한다. 이는 새로운 보안 요구사항이나 기술 변화에 대응하기 위한 기반을 제공한다.



Crypto Agility

Crypto-agility는 보안 요구사항 변화에 따라 암호 알고리즘을 유연하게 교체·적용할 수 있는 능력을 의미한다. 이는 새로운 암호알고리즘이 필요할 경우 이를 신속하게 도입할 수 있도록 하는 시스템의 특성을 포함한다. 기존 암호 시스템은 특정 알고리즘과 구현에 의존적으로 설계되는 경우가 많아 알고리즘 변경 시 시스템 수정이나 재배포가 필요하다는 한계를 가진다. 이러한 구조는 보안 대응의 지연으로 이어질 수 있다. 이에 따라 crypto-agility는 기존 시스템에 미치는 영향을 최소화하면서 암호 알고리즘과 파라미터 등 다양한 암호 요소를 유연하게 교체할 수 있도록 하는 능력으로 이해된다. 이러한 특성은 변화하는 보안환경에 효과적으로 대응할 수 있는 기반을 제공하며, 장기적으로 암호 시스템의 유지보수성과 운용 효율성을 향상시키는 데 기여한다.

암호 구성 선택의 복잡성

Sandwich 플랫폼은 다양한 암호 기능을 유연하게 지원하기 위해 프로토콜과 암호 라이브러리 조합에 따라 내부적으로 Sandwich Backend가 구성되도록 설계되어 있으며, 개발자는 이를 Sandwich API를 통해 일관된 방식으로 사용할 수 있다. 이러한 구조는 서로 다른 암호 구현을 통합된 인터페이스로 활용할 수 있게 해 다양한 환경에서의 적용 가능성을 높인다. 그러나 실제 암호 구성을 설계하는 과정에서는 프로토콜, 알고리즘, 암호 라이브러리를 함께 고려해야 하며, 각 요소의 선택에 따라 시스템의 동작 방식과 특성이 달라진다.

특히 PQC 마이그레이션 환경에서는 기존 Classical 알고리즘뿐 아니라 PQC 및 Hybrid 알고리즘까지 동시에 고려해야 하므로 가능한 구성 수가 크게 증가한다. 이에 따라 보안 수준, 성능, 자원 사용량 등을 종합적으로 고려한 설계가 요구된다. 또한 각 구성에 대한 성능·보안·자원 사용 특성의 정량적 비교 기준이나 표준화된 가이드라인이 충분하지 않아 특정 환경에 적합한 구성을 선택하기 어렵다. 동일한 기능이라도 선택된 프로토콜과 암호 라이브러리에 따라 성능과 자원 사용 특성이 달라질 수 있으며, 이를 사전에 체계적으로 판단하기 쉽지 않다. 이로 인해 구성 선택이 개발자의 경험이나 직관에 의존할 가능성이 높아지고, 시스템 간 설정 차이나 운영 환경에 따른 비밀관성이 발생할 수 있다. 결과적으로 Sandwich 플랫폼은 높은 유연성을 제공하는 동시에 암호 구성 선택의 복잡성을 증가시키며, 이러한 한계는 PQC 전환 환경에서 더욱 두드러진다.

개선 방안

정책 기반 초기 암호 구성 관리

구성 선택의 복잡성을 완화하기 위해 프로토콜과 암호 라이브러리 조합을 정책 기반으로 관리하는 방식이 필요하다. 이는 개별 요소를 직접 선택하기보다 요구사항에 따라 적절한 구성이 자동으로 결정되도록 하는 접근이다. 성능 요구사항, 보안 수준, PQC 적용 여부 등을 기준으로 정책을 정의하고, 이에 따라 적절한 알고리즘과 라이브러리 조합이 선택되도록 할 수 있다. 이 과정에서 해당 구성에 적합한 Sandwich Backend가 함께 구성되며, 개발자는 세부 설정 부담 없이 암호 기능을 활용할 수 있다. 또한 정책은 운영 환경과 서비스 특성에 따라 확장 가능하도록 설계될 수 있으며, 새로운 알고리즘이나 라이브러리가 추가되더라도 기존 정책 구조 내에서 일관되게 반영할 수 있다. 이러한 정책 기반 접근은 구성 선택 과정을 단순화하고 운영 환경에 적합한 일관된 암호 구성을 유지하는 데 기여할 수 있다.

자원 인지 기반 동적 구성 조정

초기 정책에 따라 구성된 암호 방식이라도 실제 실행 환경의 자원 상태에 따라 성능 저하가 발생할 수 있다. 이를 보완하기 위해 시스템 자원 상태를 반영하여 내부 동작을 조정하는 방식이 필요하다. 이는 메모리 상태, 네트워크 대역폭 등의 정보를 기반으로 프로토콜이나 알고리즘 구성을 조정하고, 그에 따라 Sandwich Backend가 구성되도록 하는 접근이다. 예를 들어 자원이 제한된 환경에서는 연산 부담이 낮은 구성을 적용하고, 자원이 충분한 환경에서는 보안성이 높은 구성을 선택할 수 있다. 또한 이러한 조정은 정적인 설정이 아니라 실행 환경 변화에 따라 동적으로 이루어질 수 있으며, 이를 통해 다양한 시스템 조건에서도 안정적인 성능을 유지할 수 있다. 결과적으로 이 방식은 성능 저하 가능성을 완화하고 자원 활용의 효율성을 높이는 데 기여할 수 있다.

구성 템플릿 기반 표준화 적용

구성 선택의 부담을 줄이기 위해 검증된 암호 구성 조합을 템플릿 형태로 정의하여 사용하는 방식이 필요하다. 이러한 템플릿은 프로토콜, 알고리즘, 암호 라이브러리 조합을 하나의 구성 단위로 정의하며, 해당 구성에 대응되는 Sandwich Backend를 포함한다. 개발자는 템플릿을 활용함으로써 개별 요소를 직접 조합하지 않고도 안정적인 암호 구성을 적용할 수 있다. 또한 템플릿은 사전에 검증된 조합을 기반으로 하기 때문에 구성 간 상호작용으로 인한 예기치 않은 성능 저하나 오류 발생 가능성을 줄일 수 있다. 동일한 템플릿을 반복적으로 사용할 수 있어 시스템 구성의 일관성과 재현성을 확보할 수 있으며, 운영 과정에서의 관리 및 유지보수 부담을 줄이는 데에도 기여할 수 있다.

결론

Sandwich는 다양한 암호 알고리즘과 구현을 유연하게 조합할 수 있는 구조를 제공하지만, 이러한 유연성은 프로토콜·알고리즘·암호 라이브러리를 동시에 고려해야 하는 환경에서 구성 선택의 복잡성을 증가시키는 요인으로 작용할 수 있음을 확인하였다. 또한 이러한 한계를 완화하기 위한 방안으로 정책 기반 암호 구성 관리, 자원 인지 기반 동적 구성 조정, 구성 템플릿 기반 표준화 적용을 제안하였다. 제안된 접근은 암호 구성 선택의 부담을 줄이고 다양한 운영 환경에서 일관성과 효율성을 유지하는 데 기여할 수 있다. 향후 연구에서는 제안된 개선 방향을 실제 시스템에 적용하여 성능과 적용 가능성을 평가할 필요가 있다.

